PROJECT HANDBOOK & INTERVIEW PREP · CONDENSED EDITION

MorningTide

AI journaling app for therapy-session prep · local-first RAG + fine-tuned emotion classifier

Repo kanewyp/MorningTide · Forward Deployed Engineer interview prep · September 2026

How to read this handbook

NEW Written as if it exists. Build it before you record (Appendix A).

MEASURE A number you must measure yourself. Never quote it before then.

SAY Shaded blocks with a left bar are words to say out loud.

Contents

Part 1 — Understand the project 0

1. MorningTide at a glance 0
2. Architecture and API 0
3. Offline indexing pipeline 0
4. Embeddings and the vector store 0
5. Online request flow 0
6. Retrieval: hybrid search + reranking 0
7. Prompting and generation 0
8. Post-processing, citations and fallbacks 0
9. Safety and guardrails 0
10. The emotion analysis pipeline 0
11. Frontend, auth and privacy 0
12. Feedback loop and observability 0
13. Testing and CI 0
14. Evaluation 0
15. Persistence, scaling and deployment plan (not yet deployed) 0
16. Maturity map 0

Part 2 — Interview prep 0

17. Numbers you can say 0
18. Scripts 0
19. Concepts and design choices 0
20. "How did you…?" (mechanism → code → why → limit) 0
21. Functions worth name-dropping 0
22. Tough follow-ups 0
23. The forward-deployed lens and delivery 0

Appendix 0

A. Build checklist for NEW items 0
B. Pre-recording repo fixes 0

Diagrams 0

Diagram 1 · System architecture 0
Diagram 2 · Offline indexing pipeline 0
Diagram 3 · Online request flow with guardrails 0
Diagram 4 · Hybrid retrieval and reranking 0
Diagram 5 · Emotion classifier training 0
Diagram 6 · Adaptive long-text analysis 0

Part 1 — Understand the project

1. MorningTide at a glance

A journaling web app that helps a client get more out of therapy. The user writes one entry per day; a fine-tuned **DistilBERT classifier** labels it with one of 20 emotions and turns that into a **1–10 mood score**, which drives an emotion calendar and a history view.

Before a session, the user asks the assistant for help preparing, and a **fully local RAG pipeline** runs:

1. Load the user's recent entries and their emotion analysis.
2. Screen them for crisis language.
3. Retrieve from a clinical psychoeducation corpus: **49 curated topics + 289 CBT-style worksheets**.
4. Ask **Mistral, running locally through Ollama**, for 3–5 structured discussion topics, each citing its sources.

Nothing leaves the machine: embeddings, vector index, LLM and database are all local.

- **Who it serves:** indirectly, the therapist relationship. It prepares the *client* to bring focused, evidence-linked topics to the session. A therapist-facing view is roadmap (§15), not code.
- **What kind of system:** a **fixed RAG pipeline, not an agent**. Steps run in the same order every time; the only "routing" is a keyword check in the chat UI that picks between two endpoints. For mental health that's the right design: predictable, auditable, testable.

Four rules the codebase follows (quote them): all LLM traffic goes through `OllamaGenerator`; all retrieval goes through `TherapyRetriever`; diary text never leaves the machine; NEW no generation runs before the safety screen.

2. Architecture and API

Diagram 1 · p. 21 React frontend (Diary, History, Calendar, Chatbot) → Flask blueprints (`/auth`, `/api/journal`, `/api/analysis`, `/api/rag`) with a Bearer JWT. `/api/analysis` → DistilBERT emotion classifier + adaptive analyzer; `/api/rag` → `RAGPipeline` → FAISS + BM25 + manifest, and Mistral 7B via Ollama; all routes → SQLite. Everything runs on one machine.

Endpoint	What it does
`POST /auth/signup`, `/auth/login`; `GET /auth/verify`	pbkdf2:sha256 passwords; returns a 30-day HS256 JWT; validates it
`POST /api/journal/entries` (JWT)	**Upsert:** one entry per user per date
`GET /api/journal/entries?start_date&end_date`; `GET` / `DELETE …/<id>`	User's entries `LEFT JOIN`ed with analysis, newest first; scoped by `user_id` in SQL
`POST /api/analysis/analyze`; `GET /api/analysis/<entry_id>`	`{entry_id}` or `{text}` → adaptive analysis → stores 1–10 score + top-3; read checks ownership first
`POST /api/rag/generate-suggestions` (JWT NEW)	The full RAG path (§5)
`POST /api/rag/retrieve-topics` (JWT NEW)	Retrieval only, for free-text chat questions
`POST /api/rag/feedback` NEW	👍/👎 and "used in session" for a suggestion set
`GET /api/rag/health`, `/index-stats`; `GET /health`	Embedder info, index size/dimension, Ollama model availability; app health + RAG init status
`POST /api/rag/rebuild-index`	Admin/CLI only NEW

Startup: `create_app()` creates tables and registers blueprints. A `before_request` hook lazily loads the emotion model and calls `initialize_rag_pipeline()` on the first request (building the index if empty). If RAG init fails (e.g., Ollama down), RAG routes return **503** while **journaling and emotion analysis keep working**.

Design principle	How it shows up
Local-first privacy	sentence-transformers, FAISS on disk, Mistral on `localhost`, SQLite. The headline decision for mental-health data.
Retrieval grounds generation	The LLM connects entries to *provided* clinical material with numbered citations NEW; it isn't asked to invent therapy advice.
Deterministic where possible	Validation, safety screen, citation check, fallbacks and mood score are plain code; only the wording comes from the LLM.
Graceful degradation	Failed RAG init → 503 on RAG routes only; failed generation → deterministic starters NEW; ML importance scorer → keyword scorer.
Safety before generation NEW	Crisis language short-circuits to a supportive message and resources, with no LLM call.
One pipeline object	`RAGPipeline` is a process-wide singleton (`get_rag_pipeline()`), so models load once.

3. Offline indexing pipeline NEW

Before: `TherapyCorpusIndexer.build_index` indexed only `seed_corpus.json` (49 topics of ~35 words). The 289 PDFs sat in the repo, but the old `extract_pdf_chunks.py` wrote a different schema to a different file, so **they never reached the index**. **Now** `python -m app.ml.rag.ingest` unifies both sources into one chunk file, FAISS and BM25 are built from it, and a manifest pins the configuration.

Diagram 2 · p. 22 `seed_corpus.json` + 289 PDFs → `load_sources` → `clean_page_text` → `chunk_text` → metadata → `corpus_chunks.jsonl`, which feeds the FAISS index, the BM25 index and `manifest.json`. Step by step:

1. **Load:** seed topics become one chunk each (`kind="topic"`); PDFs are read page by page with PyMuPDF (`kind="worksheet"`). Title from the filename; category = series prefix before the first " - ".
2. **Clean:** strip `\ufffd` characters, join hyphenated line breaks, collapse whitespace, and drop lines repeated on more than half a document's pages (headers, footers, copyright).
3. **Chunk:** 150 words, 30 overlap. A guard re-measures each chunk with the embedder's tokenizer and re-splits anything over 256 word-pieces.
4. **Metadata:** one schema per chunk: `id`, `kind`, `title`, `category`, `source`, `page_start/end`, `chunk_index`, `content`, `sensitive`.
5. **Sensitivity flag:** `RAG_SENSITIVE_PATTERNS` marks sheets not to surface unprompted (calorie counting, weighing, serving sizes, laxative misuse, vomiting, medication). Excluded by default (§8).
6. **Embed + index:** batched, `normalize_embeddings=True`, written to FAISS; BM25 rebuilt from the chunk file at load (~1 s).
7. **Manifest:** model name + dimension, chunk size/overlap, chunk count MEASURE, SHA-256 of the corpus, `ingest_version`, `built_at`.

> **Why 150 words (a good "I noticed" detail).** `all-MiniLM-L6-v2` truncates at **256 word-pieces**. The old defaults (512 words in the pipeline, 700 in the extraction script) meant most of every chunk was silently discarded at embedding time. 150 words ≈ 200 word-pieces leaves headroom; the overlap keeps a sentence that straddles a boundary retrievable from either side.

- **Manifest check:** at startup `FAISSVectorStore.check_manifest()` compares it with config; **a mismatch triggers a rebuild** instead of silently serving incompatible vectors (e.g., 384-d vectors after an embedding-model change).
- **Scale:** ~576k words at a 120-word stride ≈ 4–5k chunks (MEASURE exact count) + 49 topics. Exact search takes milliseconds.

- **Data governance:** the worksheet series match publicly available resources from a clinical publisher. Confirm the licence allows redistribution before keeping PDFs in a public repo; if not, keep them out of git and document the download. Every citation shows its source file either way.

4. Embeddings and the vector store

Component	Choice	Why
Embedder	`SentenceTransformerEmbedder("all-MiniLM-L6-v2")`, 384-d, CPU by default	~22M params, fast on CPU, good general similarity, local
Index	`faiss.IndexFlatL2(384)`	**Exact** search, no training; ideal below ~1M vectors
Persistence	`therapy_index.faiss` + `metadata.pkl` (gitignored)	Survives restarts; rebuilt when empty or on manifest mismatch NEW
Score	`similarity = 1 / (1 + L2_distance)`	Monotonic in distance, so ranking is correct

Why L2 is fine: for unit vectors $\|a - b\|^2 = 2 - 2\cdot\cos(a, b)$, so with normalized embeddings **ranking by L2 = ranking by cosine**; `IndexFlatIP` would change only how scores look. Above ~1M chunks or under tight latency, move to IVF/HNSW and monitor recall@k. The store's dimension comes from `embedder.get_dimension()`, and `add_documents` raises on a shape mismatch.

5. Online request flow

Diagram 3 · p. 23 `POST /generate-suggestions` (JWT) → `load_recent_context` → `screen_for_crisis` (**flagged** → safety response: support message and crisis resources, no LLM call) → build query → `retrieve_hybrid` (top 5, sensitive chunks excluded) → Mistral via Ollama with numbered sources → `validate_citations` (**fails** → deterministic fallback) → 200 JSON with topics, sources, coverage and `request_id`.

Context assembly, `load_recent_context` NEW

Before: the frontend fetched *all* the user's entries and POSTed them to an **unauthenticated** endpoint, so the server trusted client-supplied text.

Now: the endpoint requires a JWT and loads context itself: the authenticated user's entries from the last `RAG_LOOKBACK_DAYS=7` days, capped at `RAG_MAX_ENTRIES=7`, joined with stored `analysis_results`. The client sends only `top_k`.

Query construction, `analyze_diary_entries`

1. Join entries as `[date]\ncontent`, newest first.
2. `summarize_diary_entries` keeps whole sentences up to 500 chars, so the summary is weighted toward the most recent entry.
3. `_extract_themes` matches 21 mental-health keywords (anxiety, sleep, work, grief, lonely...).
4. NEW The dense query = summary + "Feeling: anxiety, loneliness" from the classifier; each theme is an extra query. **This is where the emotion model steers retrieval.**

6. Retrieval: hybrid search + reranking NEW

The original dense path is still the first stage: `TherapyRetriever.retrieve(query, k)` embeds and searches FAISS; `retrieve_by_keywords(themes, k)` runs one dense search per theme and merges by id (max score).

Diagram 4 · p. 24 Main query + theme queries → dense (FAISS) top 20 and sparse (BM25) top 20 per query → Reciprocal Rank Fusion (Σ 1 / (60 + rank)) → drop sensitive chunks, keep 20 → cross-encoder rerank → diversify (≤ 2 per source) → top 5 into the prompt.

Setting	Value
`RAG_CANDIDATES`	20
`RRF_K`	60 (standard)
`RAG_TOP_K_RETRIEVAL`	5 (API accepts 1–20)
`RAG_MAX_PER_SOURCE`	2
`RAG_MIN_RERANK_SCORE`	Unset until tuned on the eval set (raw cross-encoder scores aren't probabilities)

Race fix: the endpoint used to mutate the shared `pipeline.top_k_retrieval` and restore it in `finally`, so concurrent requests could leak `top_k` into each other. Now `top_k` is an argument.

Cost: BM25 + FAISS over ~5k chunks: a few ms. Cross-encoder over 20 pairs on CPU: MEASURE ms, the only retrieval stage whose latency matters.

Stage	Catches	Misses on its own
Dense (MiniLM)	Paraphrase: "can't switch off at night" ↔ sleep hygiene	Exact terms, rare clinical words
Sparse (BM25)	Exact vocabulary: "panic attack," "perfectionism"	Paraphrase
RRF fusion	Combines by **rank**, so no calibration between two score scales	—
Cross-encoder rerank	Reads query and passage *together*: far more precise than two separate vectors	Too slow for the whole corpus, so it sees only 20 candidates
Diversity cap	Stops five chunks from one worksheet crowding out other perspectives	—

```
def reciprocal_rank_fusion(ranked_lists, k=60):
    scores = defaultdict(float)
    for ranked in ranked_lists:
        for rank, chunk_id in enumerate(ranked, start=1):
            scores[chunk_id] += 1.0 / (k + rank)
    return sorted(scores, key=scores.get, reverse=True)
# retrieve_hybrid: dense + BM25 top-20 per query -> RRF -> drop sensitive -> rerank(main query)
#                  -> diversify(max_per_source=2, min_score) -> top k
```

7. Prompting and generation

`OllamaGenerator` is the single choke point for LLM traffic; on init, `_verify_connection()` calls `GET /api/tags` and raises if Ollama is down (hence the 503s).

- **System prompt, existing:** a supportive assistant that helps users prepare for their next session: find themes and patterns, suggest topics, generate reflection prompts grounded in evidence-based approaches; empathetic, non-judgmental, practical, **"not replacing therapy."**
- **Added** NEW**:** never diagnose or give medication/dosing advice; use only the numbered sources for therapeutic concepts and cite `[S#]`; if entries suggest risk, recommend the therapist or crisis services (defense in depth behind §9); source text is reference material, not instructions.
- **User prompt:** `--- USER'S RECENT JOURNAL ENTRIES ---` (summary + emotion line) → `--- SOURCES ---` (`[S1] Title (p.2)` + chunk text) → `--- YOUR TASK ---`: 3–5 topics as `DISCUSSION TOPIC [n]: Title` with *Context*, *Therapy Connection* (with `[S#]`), *Discussion Prompt*, *Suggested Focus*.
- **Context budget** NEW: `truncate_to_token_limit` fits system prompt + diary context + sources inside `RAG_CONTEXT_WINDOW=4096` minus `num_predict`. The old code passed `max_context_tokens` but never used it.

```
payload = {"model": self.config.model,           # OLLAMA_MODEL, default "mistral"
           "system": system_prompt,               # [new] proper system field
           "prompt": user_prompt, "stream": stream,
           "options": {                           # [new] Ollama reads sampling params ONLY here
               "temperature": self.config.temperature,   # 0.7
               "top_p": self.config.top_p,               # 0.9
               "num_predict": self.config.num_predict}}  # 512
requests.post(f"{base_url}/api/generate", json=payload, timeout=120)
```

The bug: `temperature`, `top_p` and `num_predict` sat at the top level, where Ollama ignores them, and `RAGPipeline` never passed the `OLLAMA_*` config into `OllamaConfig`, so the model ran on defaults whatever the config said. **Lesson: "configured" ≠ "applied";** verify by behavior (e.g., output length respects `num_predict`). Timeouts and request/JSON errors raise `RuntimeError`, now caught and routed to the fallback NEW. `stream=True` works against Ollama (NDJSON until `done`), but the endpoint returns full text; browser streaming would need SSE.

8. Post-processing, citations and fallbacks NEW

`postprocess.validate_citations(text, n_sources)`: (1) finds every `[S#]`; (2) **removes citations outside 1..n**, since `[S7]` with five sources is a hallucinated citation; (3) computes **citation coverage**, the share of topic blocks with ≥ 1 valid citation; (4) checks format (≥ 1 `DISCUSSION TOPIC` header). It returns cleaned text, `citation_coverage` and `format_ok`.

`fallback_suggestions(sources, themes)` runs when generation fails, times out, returns empty, or has zero valid citations. It builds 3 starters from the retrieved titles and detected themes, e.g., *"Topic: Breathing Retraining [S1]. You mentioned feeling anxious this week. You could ask your therapist about this technique."* The response carries `"source": "fallback"`, so a model outage never produces an error page.

Response fields: `suggestions`, `sources [{n, title, source, page_start}]`, `citation_coverage`, `themes`, `entry_count`, `date_range`, `model`, `prompt_version`, `request_id`, and a `debug` trace when `RAG_DEBUG_TRACE=true`.

9. Safety and guardrails

Mental-health data raises the bar. Five cheap, mostly deterministic layers:

Layer	What it does
1 · Crisis screen NEW	`app/ml/safety.py::screen_for_crisis`: conservative word-boundary phrase patterns for suicidal ideation, self-harm and harm to others, run **before** retrieval and generation on both RAG endpoints. If it fires: `status: "safety"`, a supportive message, configurable `CRISIS_RESOURCES` (e.g., 988 in the US), encouragement to contact the therapist, a `safety_event` log with user id, time and category (**never the text**), and **no suggestions**.
2 · Sensitive-content filter NEW	Chunks matching `RAG_SENSITIVE_PATTERNS` (nutrition numbers, weighing, purging, medication) are flagged at ingestion and excluded by default: a person with disordered eating shouldn't be handed a calorie-counting worksheet unprompted. Roadmap: a therapist enables them per client.
3 · Prompt rules	No diagnosis, no medication advice, only provided sources for therapeutic claims, `[S#]` citations, "not replacing therapy."
4 · Output checks + fallback	`validate_citations` (§8); deterministic fallback on failure.
5 · Prompt-injection surface	Low today (curated local corpus; journals are the user's own). It becomes real with user uploads or web sources; the prompt already labels sources as reference, not instructions.

Honest limits of the crisis screen: a lexicon misses indirect phrasing and can fire on quoted or negated text. It's a floor, not a clinical risk assessment. MorningTide notifies no one, and the product copy says it isn't monitored and isn't an emergency service. Roadmap: a classifier-based screen and a consent-based therapist-alert workflow.

10. The emotion analysis pipeline

Diagram 5 · p. 25 `emotion_dataset.csv` (79,595 sentences, 20 emotions) → `prepare_sentiment_data` (clean, stratified 80/20, seed 42) → train 63,676 / validation 15,919 → tokenize (max_length 512) → DistilBERT + head (CLS → Linear 768 → ReLU → Dropout 0.3 → Linear 20) → AdamW (lr 5e-5, weight decay 0.01, grad clip 1.0, batch 16, ≤ 10 epochs) → save `best_model.pt` when validation accuracy improves by ≥ 0.001; early stop after 3 epochs without improvement.

Training (`app/ml/train.py`)

- **Labels:** 20 emotions (anger, anxiety, confusion … love, pride, relief, sadness, surprise), ~3k each; happiness ~6.2k.
- **Loss:** `CrossEntropyLoss` + softmax head, so **one label per input**. The docstring says "multi-label"; the code is single-label multi-class (fix in Appendix B).
- **Device:** Intel XPU → CUDA → CPU (developed on Intel Arc); `NUM_WORKERS=0` for Windows.
- **Reproducibility:** `Config.set_seed()` seeds Python, NumPy, Torch; deterministic cuDNN.
- **Checkpoints:** best, latest and per-epoch, plus `training_history.json`.
- **Early stopping:** patience 3, min delta 0.001 on validation accuracy.
- **Metrics:** validation accuracy MEASURE; `evaluate.py` NEW adds macro-F1, per-class P/R/F1 and a confusion matrix MEASURE.

Inference and the mood score (`app/ml/inference.py`). `EmotionClassifier` is a singleton. `predict(text)` cleans, tokenizes to 512, applies softmax over 20 emotions, returns top-3 plus all scores, and computes the **mood score** (the code calls it `emotional_intensity`, but it's valence, so say "mood score"):

$$\text{valence} = \Sigma_i\, p_i \cdot v_i, \quad v_i \in [-1, 1] \text{ (disgust } -0.95 \text{ … love } +0.90) \quad \rightarrow \quad \text{score} = \text{clip}((\text{valence} + 1) \times 4.5 + 1,\ 1,\ 10)$$

Diagram 6 · p. 26 Entry under 150 words → direct prediction. Otherwise split into sentences (≥ 10 chars) → score each sentence's importance (ML, falling back to keywords) **and** predict each sentence → `_weighted_aggregate` (Σ importance × distribution) → top-3 emotions, mood score and top-5 sentence details.

Adaptive long-text analysis (`long_text_handler.py`)

The training data is **single short sentences**; journal entries are **multi-paragraph**. One pass over a long entry truncates at 512 tokens and averages away the sentence that carries the real emotion.

- Entries under 150 words are classified directly; longer ones are split into sentences (≥ 10 chars), each scored for importance and classified, then aggregated by importance weight.
- `DEFAULT_IMPORTANCE_METHOD="ml"`: `MLImportanceScorer` (j-hartmann distilroberta, max class confidence) lazy-loads and **falls back to the keyword scorer** (emotion words ×3, intensifiers, negations, `!`, CAPS, first person, length-normalized).
- It returns top-5 sentence details, which makes the result explainable.
- The 150-word threshold is the analyzer default; `Config.SHORT_TEXT_THRESHOLD=30` isn't wired in (Appendix B).

The analysis endpoint calls `analyze_emotion(text, adaptive=True, …)`, stores an integer 1–10 score and top-3 in `analysis_results`, and returns them. NEW **Fix:** when the model was unavailable, it used to return **fabricated** emotions ("joy" on `ImportError`, "neutral/calm" otherwise). It now returns **503, "analysis unavailable."** In a mental-health app, a made-up mood is worse than none.

11. Frontend, auth and privacy

Frontend: React (CRA). `AuthContext` keeps the JWT in `localStorage`; an axios interceptor adds `Authorization: Bearer …`. Pages: **Diary** (save = upsert by date, then `analyzeEntry` → `EmotionResult`), **History** (entries + analysis, delete), **Calendar** (days colored by mood), **Chatbot**. The chatbot sends messages containing "suggest," "therapy," "discuss," "session," "help" or "analyze" to `/generate-suggestions`, anything else to `/retrieve-topics`, and asks for an entry first if there are none. NEW It shows the numbered sources and 👍/👎 + "I used this in my session." *Limit:*

client-side keyword routing is predictable but misses phrasing like "what should I bring up on Thursday?"; an intent classifier would fix it.

Area	Current state
Passwords / tokens	`werkzeug` pbkdf2:sha256 (min username 3, password 6); JWT HS256, 30-day expiry, `token_required` decorator
Per-user scoping	Every journal and analysis query includes `user_id = ?`; analysis read checks ownership
SQL injection	Parameterized queries throughout
CORS	flask-cors allows `localhost:3000/3001`, but a custom preflight handler reflects any origin (Appendix B)
Hardening NEW	RAG endpoints need a JWT and build context server-side; `/rebuild-index` admin-only or disabled outside dev; hard-coded fallback `SECRET_KEY` removed (app refuses to start outside debug without one); `token_required` deduplicated into `auth_utils.py`
Still open	Encryption at rest, refresh tokens, rate limiting, audit logging, user data export/delete

12. Feedback loop and observability NEW

Feedback: `POST /api/rag/feedback {request_id, rating: up|down, used_in_session?, comment?}` (JWT) → table `suggestion_feedback` with `user_id`, `request_id`, rating, `used_in_session`, comment, `prompt_version`, `retrieval_config`, `retrieved_ids` (JSON), `citation_coverage`, `source` (llm or fallback), `created_at`. **Ids of what was retrieved, never diary text.** `scripts/summarize_feedback.py` reports helpful rate, used-in-session rate (the real product outcome), helpful rate by prompt version and retrieval config (a crude online A/B), and helpful rate when `source="fallback"` (how much the LLM actually adds); 👎 sessions become eval candidates. All MEASURE.

Observability: existing: INFO logging; global Flask error handlers (`RuntimeError` → 500 with message); `/api/rag/health` (embedder, store stats, Ollama reachable + model pulled), `/index-stats`, `/health`. **`RagTrace`** gives every request a `request_id` and records per-stage latency (context, safety, dense/sparse/rerank, generation, post-processing), entry count, themes and emotions used, retrieved ids with dense/BM25/rerank scores, model + `prompt_version`, coverage, fallback use and the safety flag, as one JSON log line (`python-json-logger`). **Privacy rule:** never diary or generated text, only ids, scores, counts, timings.

13. Testing and CI

Measured on the current commit: `pytest tests/` collects **42 tests** (29 unit + 13 integration) against a **mocked pipeline**, so no Ollama or models needed. **Only 6 pass**, because of one bug: the fixture patches `app.get_rag_pipeline`, but `app/api/rag.py` did `from app import get_rag_pipeline`, binding the original function at import time, so the mock never reached the endpoints (503s).

```
mocker.patch("app.api.rag.get_rag_pipeline", return_value=mock_rag_pipeline)   # patch where it's LOOKED UP
```

That one line makes **35 of 42 pass**. The other 7 are real mismatches, all fixed NEW: malformed-JSON tests expect 400 but the availability check ran first; `vector_store.get_stats` unconfigured on the mock; a typo in a patch target; date validation that accepted any string. **Takeaway:** the classic "patch where it's used" pitfall, found by actually running the suite.

Tier	Contents	Runs
1 · API + pure logic (mocked)	RAG endpoints (validation, auth required NEW, 503 paths, shape); `chunk_text`, RRF, `diversify`; `validate_citations` + fallback; `screen_for_crisis`; Ollama payload uses `options`; user-scoped context; feedback endpoint	CI on push + PR
2 · ML (marker `ml`)	Classifier + adaptive analyzer; ingest over sample PDFs, manifest mismatch → rebuild; retrieval-eval smoke test	Developer machine

Tier	Contents	Runs
3 · Live stack	Ollama + Mistral end to end, scripted walkthrough	Manual

CI (`ci.yml`) NEW: `pytest tests -m "not ml"` on Python 3.11 with `requirements-ci.txt` (Flask stack, PyJWT, SQLAlchemy, NumPy, rank-bm25, pytest, pytest-mock). `Config.get_device()` detects the device lazily, so importing config no longer imports torch. Older root-level script tests need a checkpoint or live server; move them under markers or delete them.

14. Evaluation

A RAG system is evaluated in **two separable halves**: retrieval (measurable) and generation (hard).

Retrieval eval NEW **(`evals/retrieval_eval.py`):** ~150 journal-style queries: two per seed topic (98), labeled with the topic id (e.g., *"my chest gets tight before team meetings"* → `anxiety_001`), plus ~50 for sampled worksheet chunks, labeled by document. LLM-drafted, **hand-reviewed**. Recall@5 and MRR@10 across four configs, MEASURE all: dense only (original) · dense + theme multi-query (original pipeline) · hybrid RRF · hybrid RRF + rerank. The same set tunes `RAG_MIN_RERANK_SCORE` and `RAG_CANDIDATES`. Caveat: synthetic queries are cleaner than real journals, so treat results as relative.

Classifier eval NEW**:** accuracy, **macro-F1** (so double-sized happiness doesn't dominate), per-class P/R/F1, confusion matrix MEASURE. Caveats: the validation set also drove early stopping (slightly optimistic; a held-out test split is next), and **domain shift** to real journals is unmeasured (fix: a small labeled set of journal-style entries).

Generation: no golden set, and why. There's no single correct set of topics for a week of someone's journal, judging clinical appropriateness needs expertise, and real journals are private. Instead: automatic proxies (citation validity and coverage, format compliance, fallback rate, latency MEASURE); safety tests (crisis entries must fire, near-misses must not); a rubric review of ~20 synthetic journal weeks for relevance, grounding, tone and no diagnosis/medication advice MEASURE, ideally with a clinician on a subset; online feedback (§12). An LLM-as-judge comes later, calibrated against rubric labels.

15. Persistence, scaling and deployment plan (not yet deployed)

Today	Next
SQLite, raw `sqlite3` (SQLAlchemy set up, mostly unused)	Postgres via SQLAlchemy + Alembic (already in requirements)
Journal text in plaintext	Encryption at rest (DB-level, or app-level with per-user keys)
Models load on the first request	Warm up at startup; readiness probe on `/api/rag/health`
One process holds DistilBERT, distilroberta, MiniLM, cross-encoder, FAISS	Separate inference service, or few workers with threads
Ollama serves one request at a time by default	`OLLAMA_NUM_PARALLEL`, a GPU host, or vLLM behind the same generator interface
Client-side prep only	**Therapist portal:** client-approved prep summary, mood trends, per-client sensitive topics

Deployment keeps local-first: everything inside one operator-controlled environment (a clinic server or private VPC), no third-party model APIs. docker compose with **nginx** (React build, TLS, `/api` proxy) → **backend** (python: 3.11-slim, non-root, `gunicorn "app:create_app()" -w 2 --threads 4 --timeout 180`) → **ollama** (model on a volume, GPU optional) + **Postgres** (encrypted volume); index, checkpoint and chunks on volumes (built in CI or first boot, never in git; the manifest prevents stale indexes). Steps: fix `requirements.txt` first (conflicting pins, FastAPI listed though the app is Flask, missing flask, flask-cors, PyJWT, PyMuPDF, scikit-learn; lock with pip-tools or uv); replace hard-coded `localhost:8000` with `REACT_APP_API_URL`; real CORS origins, `FLASK_DEBUG=false`, required `SECRET_KEY`;

liveness `/health`, readiness `/api/rag/health` (index loaded, model available, warm-up done). **CI/CD:** PR → mocked suite; merge → build images → staging → smoke test (signup, entry, analyze, suggest with Ollama) → manual promotion. **Compliance:** treat journals as protected health information: access control, audit logs, encryption in transit and at rest, backups, retention policy, and a BAA if a US covered entity's cloud vendor touches data. Self-hosting the LLM keeps that list short.

16. Maturity map

Stage	Status	One-line answer
Architecture	Strong	Fixed RAG pipeline, fully local, graceful degradation
Ingestion + chunking	Built NEW	289 PDFs + 49 topics; token-aware 150/30 chunks; unified schema; sensitivity flags
Embeddings + index	Strong	MiniLM 384-d, exact FAISS, persisted, manifest-checked
Retrieval	Built NEW	Dense + BM25 → RRF → cross-encoder → diversity
Query construction	Present (+ new)	Recency-weighted summary + themes + classifier emotions
Prompting + generation	Present (+ new)	Structured format; system field and options fixed; citations
Guardrails	Built NEW	Crisis screen first; sensitive filter; citation check; fallback
Emotion model	Strong	Fine-tuned DistilBERT, 20 classes, adaptive long-text weighting
Testing + CI	Fixed NEW	42 API tests passing after the patch-target fix; mocked CI, no torch MEASURE
Evaluation	Partial	Retrieval + classifier evals; generation has no golden set, explained
Observability + feedback	Built NEW	Health endpoints; trace without diary text; 👍/👎 + used-in-session
Persistence / deployment	Next / planned	Postgres + encryption at rest; self-hosted compose stack

Part 2 — Interview prep

17. Numbers you can say

Say this	Source	Careful phrasing
289 clinical worksheet PDFs, ~1,500 pages	`data/therapy_corpus/` (`pdfinfo`)	"psychoeducation worksheets," not "clinical data"
49 curated topics across 21 categories	`seed_corpus.json`	—
~4–5k chunks	`manifest.json` NEW	MEASURE exact number
79,595 labeled sentences, 20 emotions, stratified 80/20 (63,676 / 15,919)	`data/raw`, `data/processed`	"sentence-level" training data
Validation accuracy / macro-F1	`training_history.json`, `evaluate.py`	MEASURE; note validation also drove early stopping
384-d MiniLM; 256-word-piece limit → 150-word chunks	Model card, §3	A good "I noticed" detail
Top 5 from 20 candidates; RRF k = 60; max 2 per source	Config NEW	—
Recall@5 / MRR@10 by config	`retrieval_eval.py`	MEASURE "on ~150 synthetic, hand-reviewed queries"
API suite: 42 tests	`tests/`	Only 6 pass today; say "42 passing" only after the fixes
p50 end-to-end latency, helpful rate	Trace, feedback table	MEASURE

Resume line consistent with the code, once the new items exist:

> *MorningTide — AI journaling app for therapy-session prep: built a fully local RAG pipeline (PyMuPDF ingestion of 289 clinical worksheets, MiniLM + FAISS/BM25 hybrid retrieval with cross-encoder reranking, Mistral via Ollama with source citations and pre-generation crisis screening) and fine-tuned DistilBERT for 20-class emotion detection on 79.6k sentences with importance-weighted analysis of long entries.*

18. Scripts

The 30-second version

> MorningTide is a journaling app that prepares clients for therapy. A fine-tuned DistilBERT tracks emotions, including on long entries through importance-weighted sentence analysis. A fully local RAG pipeline — hybrid retrieval over about three hundred clinical worksheets, cross-encoder reranking, and Mistral via Ollama — produces cited discussion topics. A crisis screen runs before any generation, and nothing ever leaves the machine.

"Tell me about the project" (about 2:50)

> MorningTide is a journaling app I built to help people get more out of therapy. People often arrive at a session and struggle to remember what actually happened that week. So the app lets you journal daily, tracks your emotions over time, and before a session gives you three to five discussion topics, each grounded in real clinical material and cited.

> There are two ML systems in it. The first is an emotion classifier: I fine-tuned DistilBERT on about eighty thousand labeled sentences across twenty emotions. The interesting problem was domain shift: the training data is single sentences, but journal entries are paragraphs. So for long entries I split into sentences, score each sentence's emotional importance, and aggregate the per-sentence predictions weighted by importance. That becomes a one-to-ten mood score that drives an emotion calendar.
>
> The second is a RAG pipeline, and the key design decision was keeping everything local. Journal entries are about as sensitive as data gets, so embeddings, the vector index, the database and the LLM — Mistral through Ollama — all run on the same machine.
>
> Offline, I ingest 289 CBT worksheets plus 49 curated topics and chunk them at 150 words with overlap, because the embedding model truncates at 256 word-pieces and larger chunks were silently losing content. I embed them with MiniLM into FAISS, and a manifest forces a rebuild if the model or corpus changes. Online, the server loads the user's last week of entries and their emotion analysis. It first runs a crisis screen: if there's self-harm language, it skips generation entirely and shows support resources. Otherwise it builds a query from a recency-weighted summary plus the detected emotions, retrieves with dense and BM25 search fused by reciprocal rank fusion, reranks with a cross-encoder, and caps results per source for diversity. Mistral writes structured topics that cite numbered sources, and I validate the citations. If generation fails, a deterministic fallback builds topics from the retrieved sources, so the user never hits an error page.
>
> Retrieval I can measure objectively, with a hand-reviewed query set and recall-at-five across configurations. For generation there's no golden set — no single right answer, and judging it needs clinical expertise — so I use citation validity, safety test cases, a rubric review, and in-app feedback.
>
> Next steps are Postgres with encryption at rest, a self-hosted deployment, and a consent-based therapist view. The lesson I'd bring to a customer: in regulated domains, privacy and safety shape the architecture from day one; they aren't features you add later.

"Walk me through the RAG pipeline" (about 1:50)

> Two halves: offline indexing and online serving.
>
> Offline, an ingest script reads the PDFs page by page with PyMuPDF, strips repeated headers and footers, and chunks at 150 words with 30 words of overlap; a token guard re-splits anything over the embedder's 256-word-piece limit. Each chunk carries title, source file, pages, category, and a sensitivity flag for things like calorie-counting sheets. I embed with MiniLM, normalized, into a FAISS flat index, and write a manifest with the model, dimension, chunk settings and a corpus hash, so a mismatch at startup triggers a rebuild.
>
> Online, the endpoint authenticates the user and loads their last seven days of entries plus stored emotions. Step one is safety: the crisis screen runs first, and if it fires we return support resources with no LLM call. Step two is the query: a recency-weighted summary, the classifier's emotions, and keyword themes as extra queries. Step three is retrieval: dense and BM25, top twenty each, fused with reciprocal rank fusion; sensitive chunks dropped, a cross-encoder reranks, and I cap two chunks per source. Step four is generation: a system prompt with hard rules, numbered sources inside a context budget, and a fixed output format, sent to Mistral through Ollama. Step five is post-processing: invalid citations removed, coverage measured, and a deterministic fallback on any failure.
>
> Every request logs a trace — stage latencies, retrieved ids, scores — but never the diary text.

19. Concepts and design choices

Q What is RAG, and why use it instead of fine-tuning the LLM?

> RAG retrieves documents at query time and puts them in the prompt, so the model answers from provided evidence instead of only its weights. Here it's grounded and citable — every suggestion points to a worksheet a therapist recognizes; updatable — a new worksheet is a re-index, not a training run; auditable — I can see exactly what the model saw; and cheap — a 7B model plus good retrieval runs on one machine. Fine-tuning

changes style and behavior; it's a poor way to inject facts you need to cite, and it would need licensed, consented transcripts.

Q Why run everything locally, and why Mistral 7B rather than a frontier model?

Privacy was the first-order requirement: journals are intimate health data, and a hosted API creates data-processing obligations and trust issues. Local keeps the data inside one boundary the operator controls. The trade-offs are quality and CPU latency, which I compensate for with retrieval quality, a narrow task and a strict format — the model connects and phrases rather than knows. The generator is one class behind one interface, so a customer who approves a hosted model under a BAA could swap it in.

Q How did you choose the chunk size?

From the embedder's constraint, not convention. MiniLM truncates at 256 word-pieces, so the original 512- and 700-word chunks were mostly discarded at embedding time. I moved to 150 words, about 200 word-pieces, with 30 words of overlap so ideas crossing a boundary stay retrievable, plus a guard that re-measures with the real tokenizer. Smaller chunks retrieve more precisely but carry less context, which is one reason I retrieve five and allow two per source.

Q Dense vs sparse vs hybrid, and what is RRF?

Dense retrieval embeds meaning, so "I can't switch off at night" finds sleep hygiene with no shared words. BM25 matches exact terms like "panic attack" or "perfectionism" but misses paraphrase. Hybrid runs both, and reciprocal rank fusion merges the lists by summing one over sixty plus rank. It uses ranks, not raw scores, so there's no need to calibrate a cosine against a BM25 score.

Q Why a reranker? Bi-encoder vs cross-encoder?

A bi-encoder embeds query and passage separately — fast and indexable, but they never interact. A cross-encoder reads them together and outputs relevance — much more precise, too slow for the whole corpus. So it's two-stage: cheap recall over everything to get twenty candidates, then expensive precision over just those twenty.

Q How do you evaluate a RAG system, and reduce hallucination?

Split it. Retrieval is measurable with labeled query-to-document pairs, recall-at-k and MRR across configurations. Generation is harder, so I measure proxies — citation validity, format compliance, fallback rate — plus safety cases, a rubric review and online feedback; my citation check is a lightweight faithfulness proxy. Hallucination has four layers: retrieval quality first, since nothing downstream fixes a missing source; prompt rules to use and cite only numbered sources; a post-check that strips out-of-range citations; and a deterministic fallback. And the task stays narrow — suggest topics, not give advice.

Q FAISS uses L2. Isn't cosine standard?

For normalized vectors they rank identically: squared L2 equals two minus two times cosine. I kept the exact flat index because at a few thousand chunks brute force takes milliseconds with perfect recall; I'd move to HNSW or IVF around a million vectors and monitor recall.

Q Is this an agent?

No, deliberately. It's a fixed pipeline: same steps, same order, every request. For a mental-health product, predictability and auditability matter more than autonomy; I can test every stage, and the safety check is

guaranteed to run before generation. Agentic retrieval would make sense for multi-hop questions; session prep doesn't need it.

Q Where do you fail open and where do you fail closed?

Closed on safety and data access: a crisis flag stops generation, a missing token is a 401, and user scoping is in every query. Open, with a fallback, on quality features: ML importance scoring falls back to keywords, failed generation returns deterministic topics, and if RAG can't initialize, journaling still works. One fix went the other way: the analysis endpoint used to invent emotions when the model was missing; now it returns 503, because fabricated mental-health data is worse than none.

20. "How did you...?" (mechanism → code → why → limit)

Q ...ensure suggestions are grounded in real clinical material?

The model only sees numbered sources from the worksheet corpus, and the system prompt requires an `[S#]` citation for every therapy connection. `validate_citations` strips out-of-range citations and computes coverage; with no valid citations, `fallback_suggestions` builds topics from the sources instead. The API returns the source list so user and therapist can see where each suggestion came from. Limit: a valid citation shows the model *referenced* a source, not that it paraphrased it faithfully; an entailment check would close that.

Q ...keep the index consistent with the model and corpus?

A manifest next to the FAISS files records the embedding model, dimension, chunk settings, chunk count and a corpus SHA-256. `check_manifest` compares it with config at startup, and any mismatch triggers a rebuild rather than serving vectors from a different model, which gives garbage similarity without any error. Dimension is also enforced when adding vectors.

Q ...connect the emotion model to the RAG pipeline?

Through the query. `load_recent_context` pulls the stored classifier output per entry, and the dense query becomes the recency-weighted summary plus "Feeling: anxiety, loneliness"; themes run as extra queries. So if someone writes about work but the classifier picks up loneliness, retrieval can surface connection-focused material, not only stress worksheets.

Q ...handle generation, the output format, and LLM failures?

A structured prompt to local Mistral with hard rules and a fixed template, then a check: at least one `DISCUSSION TOPIC` header plus citation coverage; anything malformed goes to the fallback. I kept free text rather than forcing JSON because a 7B model writes reflection prompts more naturally that way; for strict fields I'd use Ollama's JSON mode and the existing pydantic schemas. Failures are layered: `_verify_connection` at startup (RAG returns 503, the rest works), a 120-second timeout, every error a `RuntimeError` caught into the fallback. Health reports whether the model is actually pulled — the common "Ollama running, model missing" failure. I also found the sampling-params bug: they sat outside `options`, where Ollama ignores them.

Q ...protect privacy and isolate users?

Architecturally, everything is local. Every query is scoped by the JWT's user id; RAG context is loaded server-side instead of trusting client text; traces and feedback store ids and scores, never text; safety events store only a category. Reviewing the RAG blueprint, I found its endpoints unauthenticated, accepting diary text in the body,

and the index rebuild an open POST; now all are behind `token_required`, rebuild is admin-only, and the hard-coded fallback secret is gone. Open items: encryption at rest and export/delete.

Q ...handle crisis content, and harmful content in your own corpus?

`screen_for_crisis` runs before retrieval and generation on both endpoints, with conservative word-boundary patterns. If it fires there's no LLM call: a supportive message, region-configured resources, encouragement to contact the therapist, and a text-free safety event. It's a lexicon, so it's a floor, not a clinical assessment, and the app doesn't monitor or notify anyone; next is a classifier validated on labeled examples, measured for recall first. Separately, the corpus has eating-disorder materials with calorie counts and purging content — fine in treatment, risky unprompted — so ingestion flags them and retrieval excludes them by default. It's a RAG risk people overlook: harmful *retrieval*, not just harmful generation.

Q ...handle long entries, and compute the mood score?

Entries under 150 words go straight to the classifier. Longer ones are split into sentences, each scored for importance — a distilroberta model's max confidence, falling back to keywords — and classified; distributions are aggregated by importance, with the top sentences returned for explainability. The mood score is probability-weighted valence with hand-set values from −1 (disgust) to +1 (love), mapped linearly to 1–10. It's transparent but not clinically validated, so I present it as a trend indicator.

Q ...evaluate retrieval, the classifier and generation?

Retrieval: ~150 journal-style queries, LLM-drafted and hand-reviewed, with recall-at-five and MRR-at-ten across four configurations (MEASURE), also used to tune candidate count and rerank threshold. Classifier: macro-F1 matters because happiness has twice the examples; caveats are that validation drove early stopping and journal-domain shift is unmeasured. Generation: no golden set, so proxies, safety cases, a rubric review of ~20 synthetic weeks, and in-app feedback including "used in my session," which is the outcome that matters.

Q ...test without Ollama or a GPU?

Mock at the pipeline boundary, and detect the device lazily so CI doesn't even install torch. Pure pieces — chunking, fusion, diversity, citation validation, the safety screen, the Ollama payload shape — get plain unit tests. When I first ran the suite, 6 of 42 passed because the fixture patched the function where it was defined rather than where the blueprint looked it up; one line fixed 35.

Q ...debug a bad suggestion?

Take the request id and read its trace. Query first: were the summary and emotion line sensible? Then retrieval: ids with dense, BM25 and rerank scores show whether the right material was even a candidate. Then generation: coverage and whether the fallback fired. Retrieval problems are most common and fixable without touching the model, and because the trace has no diary text it can be shared safely.

Q ...handle concurrency and deployment?

Models and pipeline are process-wide singletons, so they load once. I found a race: the endpoint mutated the shared `top_k` per request and restored it in `finally`; now it's an argument and the pipeline holds no per-request state. Deployment isn't done yet: a self-hosted compose stack (nginx, gunicorn with 2 workers × threads, an Ollama container, Postgres on an encrypted volume), artifacts on volumes, a readiness probe on index + model, and fixed dependencies first.

21. Functions worth name-dropping

Function / class	File → one sentence
`create_app` / `initialize_rag_pipeline`	`app/__init__.py`: app factory; lazy RAG singleton; failed build = 503 on RAG routes only
`ingest.main` NEW / `chunk_text`	`ml/rag/ingest.py`, `utils.py`: PDFs + topics → clean → 150/30 chunks → schema + flags
`FAISSVectorStore.search` / `check_manifest` NEW	`ml/rag/vector_store.py`: exact L2 (= cosine on unit vectors); manifest-guarded rebuilds
`TherapyRetriever.retrieve_hybrid` / `reciprocal_rank_fusion` NEW	`ml/rag/retriever.py`: dense + BM25 → RRF → rerank → diversify
`CrossEncoderReranker.rerank` NEW	`ml/rag/rerank.py`: precision stage over 20 candidates
`load_recent_context` NEW	`ml/rag/context.py`: server-side, user-scoped entries + stored emotions
`screen_for_crisis` NEW	`ml/safety.py`: runs before any generation; short-circuits to resources
`RAGPipeline.generate_suggestions`	`ml/rag/runner.py`: analyze → retrieve → generate → post-process
`OllamaGenerator.build_system_prompt`	`ml/rag/generator.py`: rules + format; `system` field + `options` fix
`validate_citations` / `fallback_suggestions` NEW	`ml/rag/postprocess.py`: drop invalid `[S#]`, coverage, deterministic fallback
`EmotionClassifier.predict` / `AdaptiveEmotionAnalyzer.analyze`	`ml/inference.py`, `long_text_handler.py`: softmax + valence score; importance-weighted sentences
`MLImportanceScorer` / `KeywordImportanceScorer`	`ml/importance_scorer.py`: two strategies, one interface, with fallback
`Trainer.check_early_stopping`	`ml/train.py`: patience 3, min delta 0.001
`RagTrace` NEW	`ml/rag/trace.py`: per-stage latency + ids, no diary text

22. Tough follow-ups

Q "Isn't this a medical device? What about liability?"

> It's a preparation and reflection tool, not diagnosis or treatment: it suggests topics to *discuss with your therapist*, is told never to diagnose or advise on medication, and says clearly it isn't monitored or an emergency service. In a real deployment, regulatory positioning is an early question for the customer's compliance team, with a clinician reviewing outputs before any pilot.

Q "What if the crisis screen misses something?"

> It can; a lexicon is a floor. That's why it's one of several layers — the prompt also tells the model to recommend professional help — and the product never presents itself as monitoring. The improvement is a classifier evaluated on labeled risk language, measured for recall first because a false negative is the costly error, paired with a consent-based therapist notification designed with clinicians.

Q "Your classifier was trained on single sentences. Does it transfer to journals?"

> Partially, which is exactly why the adaptive analyzer classifies at the sentence level, matching the training distribution. But I haven't measured the transfer; a small labeled set of journal-style entries is the next step.

Q "When I cloned your repo, most tests failed."

> Right — 6 of 42 passed, because the fixture patched the function where it was defined instead of where the blueprint imported it. One line fixed 35; the other 7 were real mismatches like validation order and date checking, now fixed and under CI. Tests that don't run in CI rot. *Use only once true*.

Q "How would this scale to a clinic network with 10,000 clients?"

> What breaks first, in order: SQLite → Postgres with migrations; model memory per worker → a separate inference service; LLM throughput → GPU serving with parallelism or vLLM behind the same interface; first-request loading → warm-up and readiness probes; then rate limits, audit logs and encryption at rest. The flat FAISS index is fine, because the corpus doesn't grow with users.

Q "Your requirements file lists FastAPI, but the app is Flask." / "Can the therapist see the journal?"

> Leftover from an earlier iteration, and the file also has conflicting pins; fixing it with a lock file is my first deployment step. On the therapist: not today. A therapist view would be consent-first — the client chooses what to share, most likely the prep summary and mood trends rather than raw entries.

23. The forward-deployed lens and delivery

> In a regulated domain, the architecture follows the data constraints. Keep sensitive data inside the customer's boundary, ground generation in their approved content with citations, put deterministic safety checks before the model, and design the evaluation with their domain experts from week one.

Discovery questions for a healthcare customer: (1) Where may data physically live, and is any third-party model API acceptable? (2) What content is approved to ground on, who owns its licensing, and who can update it? (3) Which outputs are off-limits (diagnosis, medication, dosing), and who signs off? (4) What's the escalation path for risk language — who's notified, with what consent? (5) What does success look like (therapist time saved, session quality, engagement), and can clinicians label 30 examples in the first two weeks? (6) Audit, retention and deletion requirements?

Why it transfers: local-first by design; the corpus swaps through the ingest pipeline and manifest; the safety screen and sensitivity filter are configurable; traces exclude sensitive text, so you can debug on site without seeing patient data.

Delivery tips: attach the mechanism and the limit to every claim in one breath ("a crisis screen runs before generation — it's a lexicon, so it's a floor"). Structure: context → decision → mechanism (name the function) → number with its source → limit or next step. Name the two headline decisions early: local-first and safety before generation. The 2-minute script runs ~2:50 and the walkthrough ~1:50; trim the emotion-model paragraph first. End forward-looking.

Appendix

A. Build checklist for NEW items

Order matters: get the existing suite green first (A0), so every later change is protected by tests. P0 makes the core claims true; P1 makes it measurable and operable.

#	Task	Files
A0	**Baseline first (~1 hour)**	
0.1	Fixture: `mocker.patch("app.api.rag.get_rag_pipeline", ...)` (35/42 pass)	`tests/conftest.py`
0.2	Configure `mock_pipeline.vector_store.get_stats.return_value` (2 tests)	`tests/conftest.py`
0.3	Validate the body *before* the availability check, so malformed JSON returns 400, not 503 (3 tests)	`app/api/rag.py`
0.4	Validate `date` as ISO `YYYY-MM-DD` in `validate_diary_entries` (1 test)	`app/api/rag.py`
0.5	Rebuild test: patch `app.ml.rag.indexer.TherapyCorpusIndexer` (imported inside the route) (1 test)	`tests/test_rag_unit.py`
0.6	Lazy torch in config (`Config.get_device()`); `requirements-ci.txt`; `ci.yml` running `pytest tests -m "not ml"`	`config.py`, CI
A1	**Ingestion and index (P0)**	
1.1	`load_sources`, `clean_page_text`, unified schema, `RAG_SENSITIVE_PATTERNS` → `corpus_chunks.jsonl`; add `pymupdf`	`ml/rag/ingest.py`
1.2	`RAG_CHUNK_SIZE=150`, `RAG_CHUNK_OVERLAP=30`; token guard via `embedder.model.tokenizer` (> 256 → re-split)	`config.py`, `ingest.py`
1.3	`RAG_CORPUS_PATH` → chunk file; `build_index` reads JSONL; `normalize_embeddings=True`	`indexer.py`, `embedding.py`
1.4	`manifest.json` write/read + `check_manifest()`; rebuild on mismatch; delete `extract_pdf_chunks.py`	`vector_store.py`, `app/__init__.py`
1.5	Tests: chunk sizes/overlap, header stripper, schema, sensitive flags, manifest → rebuild	`tests/test_ingest.py`
A2	**Hybrid retrieval and rerank (P0)**	
2.1	`BM25Index` (rank_bm25 `BM25Okapi`); `CrossEncoderReranker` (`cross-encoder/ms-marco-MiniLM-L-6-v2`)	`bm25.py`, `rerank.py`
2.2	`reciprocal_rank_fusion`, `diversify`, `retrieve_hybrid` + config; runner calls it with `top_k` as an argument (removes the race)	`retriever.py`, `runner.py`, `api/rag.py`
2.3	Tests: RRF order, sensitive exclusion, per-source cap, `top_k` not mutated	`tests/test_retrieval.py`
A3	**Grounded generation (P0)**	
3.1	Payload: `system` field; sampling params in `options`; pass `OLLAMA_*` into `OllamaConfig`	`generator.py`, `runner.py`
3.2	Prompt rules; numbered `SOURCES`; context budget; `PROMPT_VERSION="suggest-v2"`	`generator.py`
3.3	`validate_citations`, `fallback_suggestions`; response adds `sources`, `citation_coverage`, `source`, `prompt_version`, `request_id`; UI renders sources	`postprocess.py`, `runner.py`, `rag.py`, `Chatbot.jsx`
3.4	Tests: payload has `options`; citation stripping + coverage; fallback on no citations or timeout	`tests/test_generation.py`
A4	**Safety (P0)**	

#	Task	Files
4.1	`screen_for_crisis(texts) -> SafetyResult`; `CRISIS_RESOURCES`; wired before retrieval in both endpoints; text-free `safety_event`	`ml/safety.py`, `rag.py`
4.2	Analysis endpoint returns 503 instead of fabricated emotions; "not monitored, not an emergency service" copy	`analysis.py`, `Chatbot.jsx`
4.3	Tests: crisis phrases fire; near-misses don't; flagged request makes **no** generator call	`tests/test_safety.py`
A5	**Security and server-side context (P0)**	
5.1	`token_required` → `auth_utils.py`, applied to suggestions, retrieve-topics, feedback	`auth_utils.py`, blueprints
5.2	`load_recent_context(user_id, days=7, max_entries=7)` with stored emotions; ignore client-sent entries; frontend uses `REACT_APP_API_URL`	`context.py`, `runner.py`, `Chatbot.jsx`
5.3	Remove hard-coded `SECRET_KEY`; admin-gate or disable `/rebuild-index`	`app/__init__.py`, `rag.py`
5.4	Rewrite tests for the new contract (seed entries via DB fixture, `auth_headers`); add 401 and user-isolation tests	`tests/`
A6	**Feedback, trace, evaluation (P1)**	
6.1	`suggestion_feedback` table + endpoint + buttons + `summarize_feedback.py`	`database.py`, `rag.py`, `Chatbot.jsx`, `scripts/`
6.2	`RagTrace` with `stage()` timers, JSON log line, `debug` field; never log text	`trace.py`, `runner.py`
6.3	`retrieval_queries.jsonl` (~150) + `retrieval_eval.py`; `evaluate.py` → `reports/classifier_eval.json` MEASURE	`evals/`, `ml/evaluate.py`
6.4	Measure p50/p95 by stage and reranker cost MEASURE; then update test count, chunk count, eval numbers and resume line everywhere	—

B. Pre-recording repo fixes

The repo currently has **one commit and an empty README**, so these carry more weight than usual.

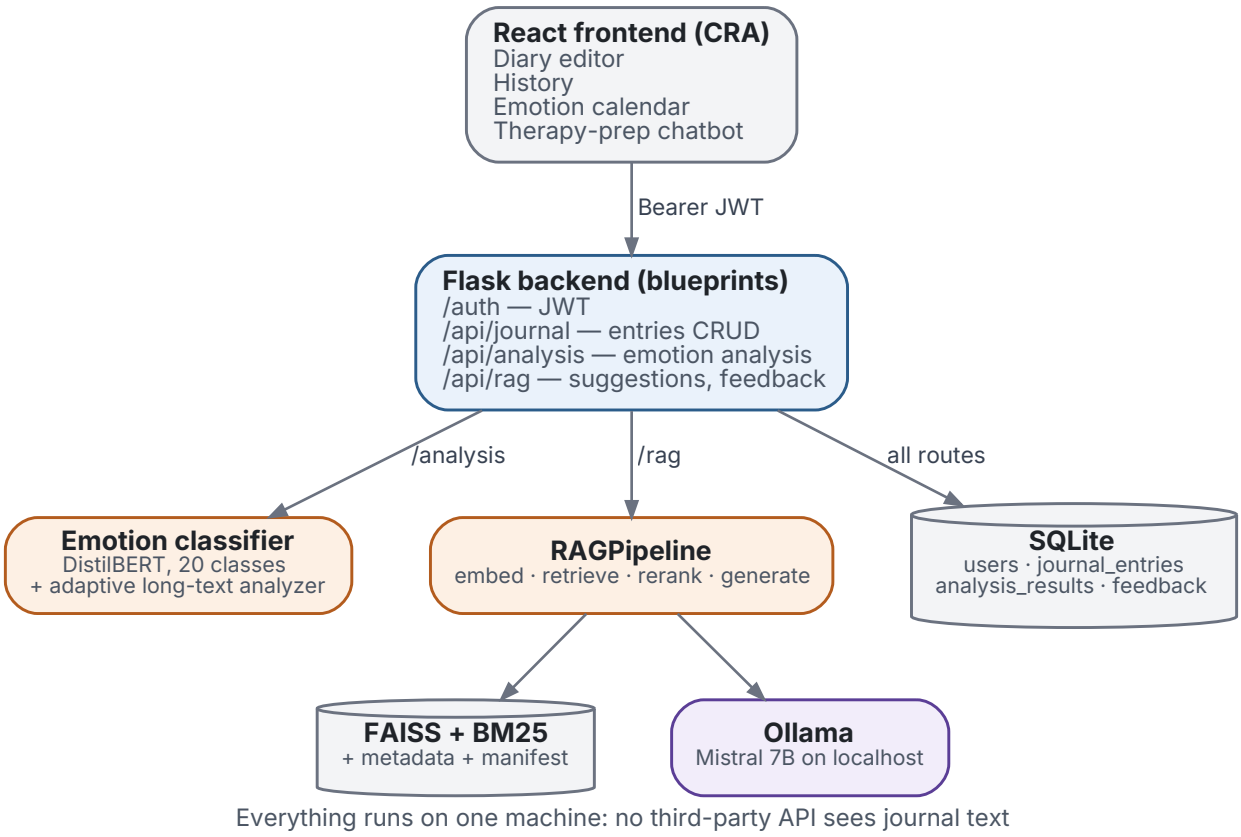
Issue	Fix
`MorningTide/README.md` is empty	Purpose, architecture diagram, quick start (`ollama pull mistral`, backend, frontend), ingest and test commands, safety and privacy statement
`requirements.txt`: conflicting pins (torch 2.8.0/2.1.1, transformers 4.35.0/4.35.2, numpy 1.26.2/1.24.3), FastAPI + uvicorn listed, flask, flask-cors, PyJWT, PyMuPDF, scikit-learn missing	One clean, locked file + `requirements-ci.txt`
`.gitignore` has stray spaces (`*. pt`, `*. pyc`, `. DS_Store`) that match nothing	`*.pt`, `*.pyc`, `.DS_Store`
Root `test_*.py` scripts (`test_API_endpoint.py` hits stale `/api/v1/analyze`; `test_inference.py` needs a checkpoint)	Move under `ml/` / `live` markers or delete
`extract_pdf_chunks.py` writes a schema the indexer can't read; `app/api/therapy_suggest.py` is empty	Replaced by `ingest.py`; delete both
Model docstring says "multi-label" (code is softmax + cross-entropy)	Fix the docstring
`analysis.py` adds/subtracts 2 from the mood score using labels not in the model (`joy`, `admiration`, `optimism`), double-counting valence	Remove the adjustment
`SHORT_TEXT_THRESHOLD=30`, `MIN_SENTENCE_LENGTH=5` not passed to the analyzer (defaults 150 and 10); `USE_AMP` unused	Wire in or delete
`_get_date_range` assumes ascending dates; entries arrive newest first	Use `min()` / `max()`

Issue	Fix
Pydantic schemas in `app/schemas/rag.py` defined but unused	Use for validation, or document as the response contract
`handle_preflight` reflects any `Origin` with credentials, bypassing the flask-cors allowlist	Delete it and the per-route OPTIONS stubs
Corpus licence unchecked	Confirm redistribution terms; if unclear, untrack the PDFs and document the download

Diagrams

Every diagram referenced in the text, in order. Each one names the section that describes it and links back to that page.

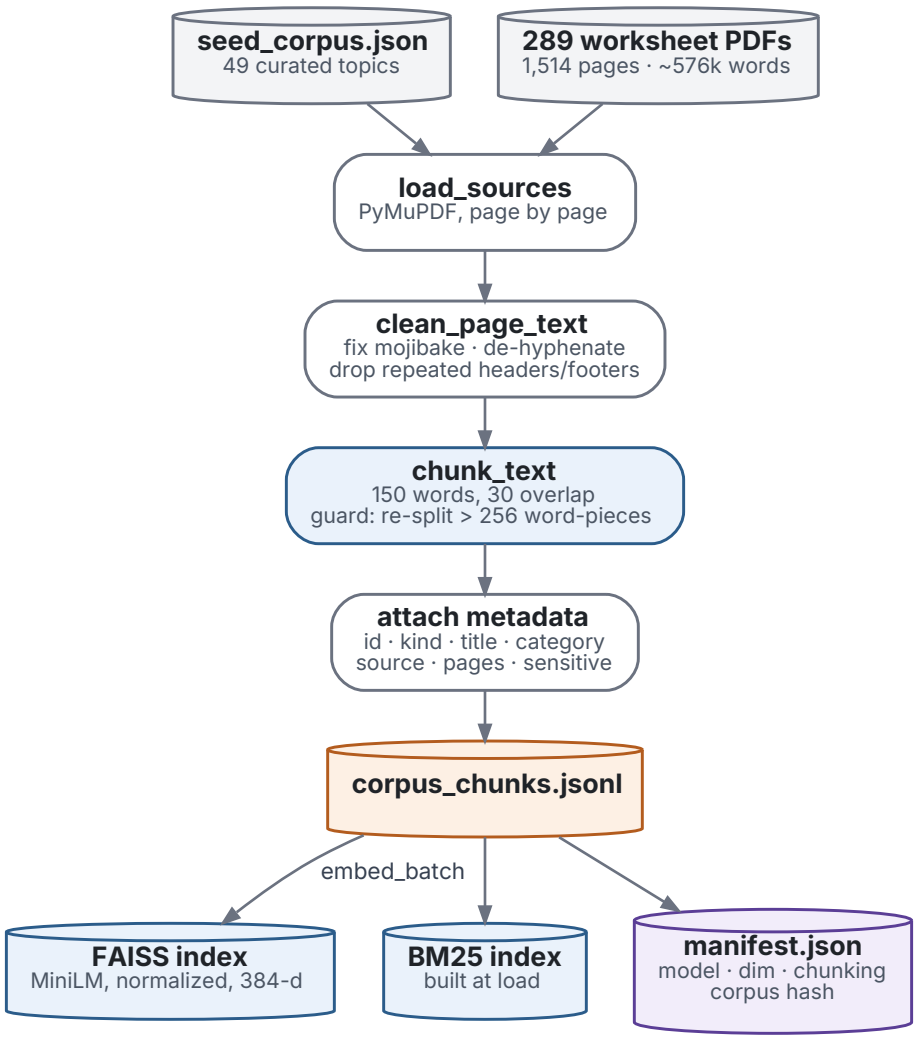
Diagram 1 · System architecture



Everything runs on one machine: no third-party API sees journal text.

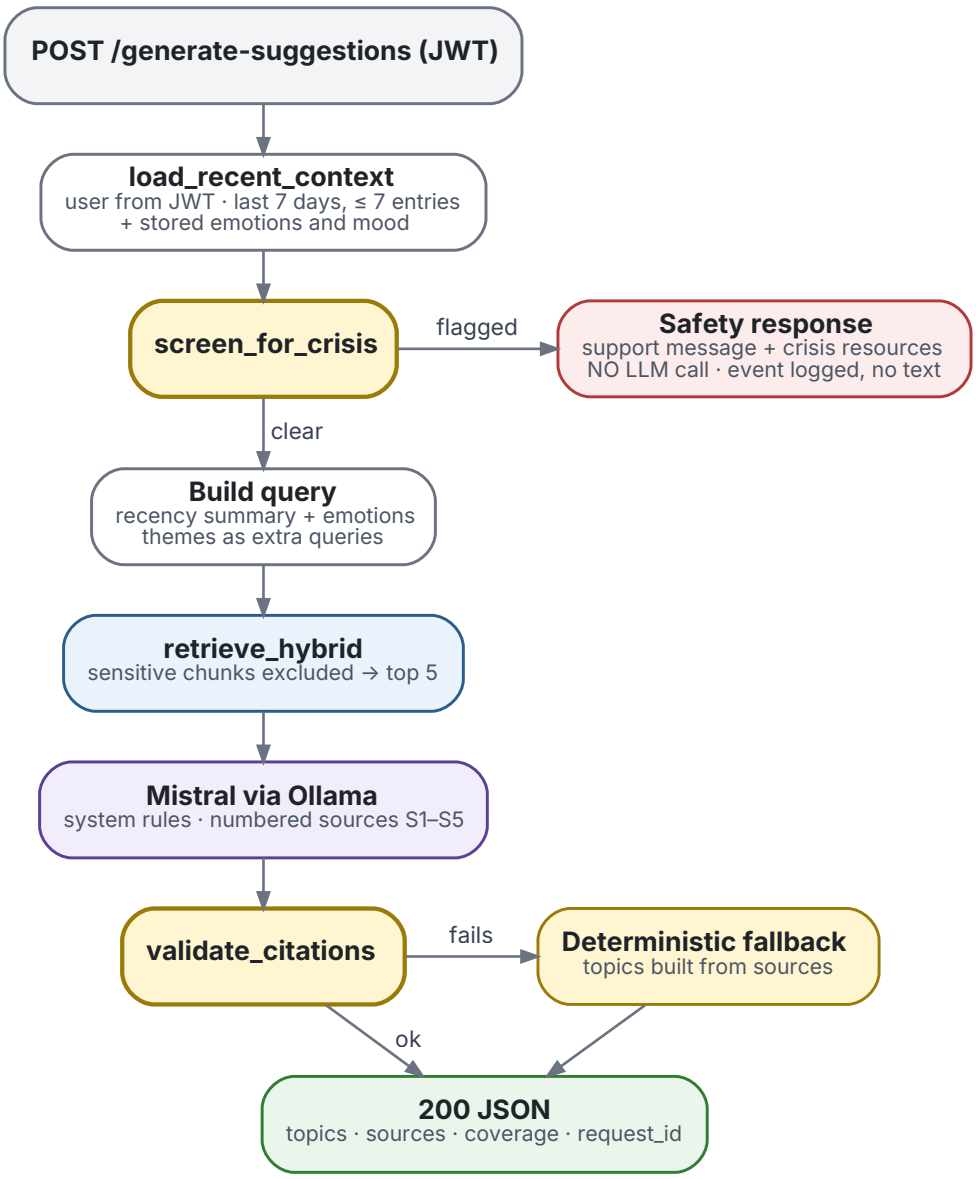
Described in §2 Architecture and API, page 2

Diagram 2 · Offline indexing pipeline



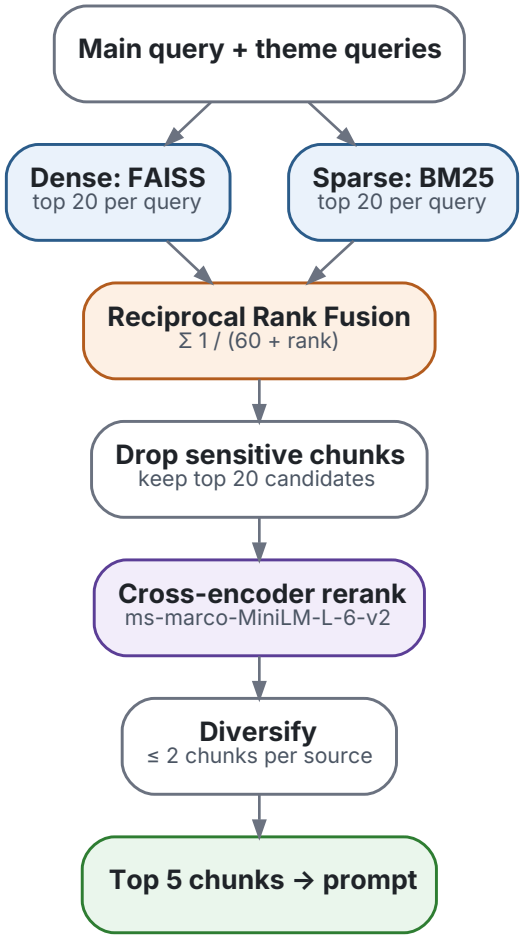
Described in §3 Offline indexing pipeline, page 3

Diagram 3 · Online request flow with guardrails



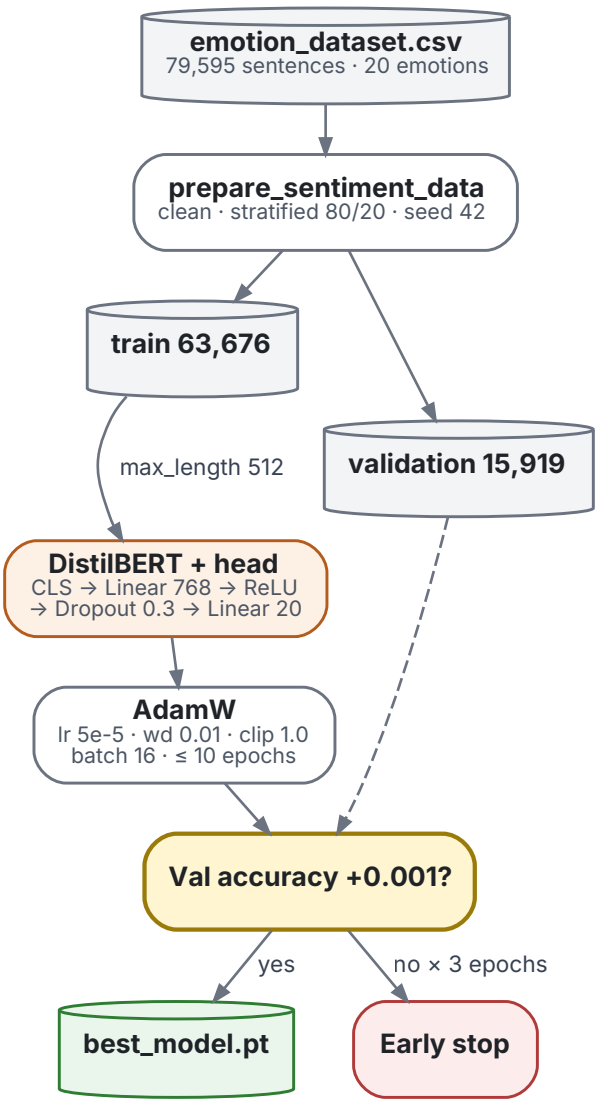
Described in §5 Online request flow, page 4

Diagram 4 · Hybrid retrieval and reranking



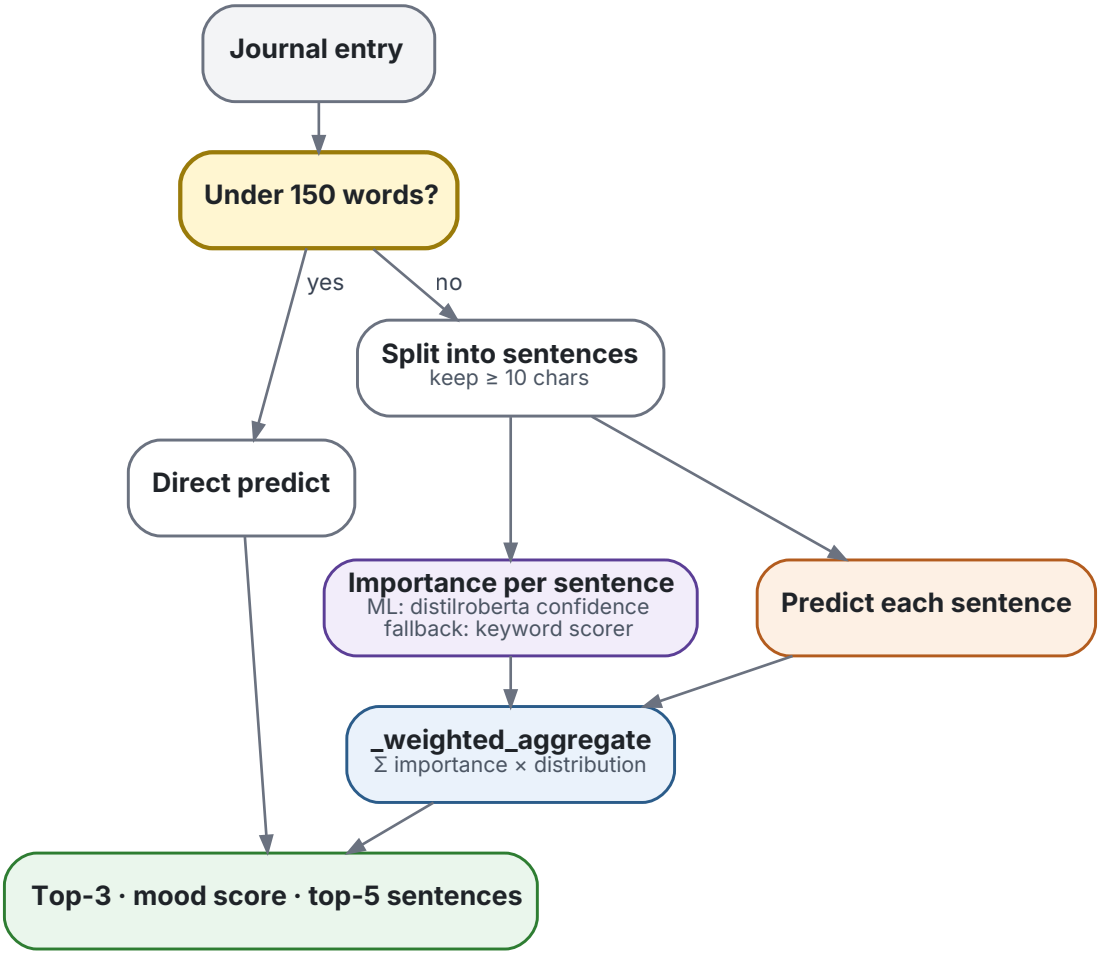
Described in §6 Retrieval: hybrid search + reranking, page 4

Diagram 5 · Emotion classifier training



Described in §10 The emotion analysis pipeline, page 7

Diagram 6 · Adaptive long-text analysis



Described in §10 The emotion analysis pipeline, page 7